

Package ‘CrossingTools’

July 12, 2026

Title Tools for Cross Optimization in Plant Breeding Programs

Version 1.0.8

Description Mate allocation is a critical component of plant breeding programs, aiming to generate superior progenies and improved populations over time. While numerous methodologies for mate allocation exist, few software tools offer a unified and scalable framework that integrates these methodologies in a user-friendly manner. 'CrossingTools' addresses this gap by providing a flexible, efficient solution for optimizing mating decisions across a wide range of breeding schemes. The package supports genomic BLUP and selection indices for various variance structures and implements multiple selection criteria to evaluate cross potential, including expected mean, optimal haploid value (OHV), and superior progeny value (SPV). Different methods are provided for calculating additive and dominance segregation variances for both inbred and heterozygous parents, making 'CrossingTools' suitable for a wide range of breeding strategies. After criterion calculation, users can rank potential crosses or apply the built-in optimal crossing selection routine, which balances genetic gain and diversity to support long-term improvement. The package combines R's scripting flexibility with high-performance C++ routines via 'Rcpp' and 'RcppArmadillo', ensuring scalability for large populations with dense marker data.

License GPL-3 | file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE, roclets = c(`\`collate`, ``\`namespace`, ``\`rd``))

ByteCompile true

RoxygenNote 7.3.3

Depends R (>= 4.0.0)

Imports ggplot2,
ggridges,
plotly,
Rcpp,
RcppParallel

LinkingTo Rcpp,
RcppArmadillo,
RcppParallel

SystemRequirements C++17

Suggests knitr,
rmarkdown,
testthat (>= 3.0.0)

Config/testthat/edition 3

VignetteBuilder knitr

URL <https://github.com/svenernstW/CrossingTools>

BugReports <https://github.com/svenernstW/CrossingTools/issues>

R topics documented:

backsolve_marker_effects	2
calc_midparent_inbred	3
calc_midparent_outcross	4
calc_optimal_haploid_value	5
calc_spv_inbred	6
calc_spv_outcross	7
make_cross_plan	8
make_index	9
optimize_cross_plan	11
plot_cross_plan	13
predict_response	14
summarize_cross_plan	15
Index	16

backsolve_marker_effects
<i>Backsolve marker effects from individual effects</i>

Description

Computes marker effects $\mu = (scalingFactor * M' * G^{-1}) * g$. The scaling factor is automatically determined from the data.

Usage

backsolve_marker_effects(marker.mat, G.mat, genotype.effects)

Arguments

- marker.mat Numeric matrix (n x p): marker/genotype matrix (individuals in rows, markers in columns).
- G.mat Numeric matrix (n x n): relationship matrix among individuals.
- genotype.effects Numeric (n x k) or length-n vector: individual effects (one or more traits).

Value

A numeric p x k matrix of marker effects .

calc_midparent_inbred *Predicted cross means (additive) from specified crosses*

Description

Compute the expected genomic estimated breeding value (EGEBV) for each proposed cross as the mean of the parental GEBVs (mid-parent value). Optionally, compute a selection index per cross as a weighted sum across traits.

Usage

```
calc_midparent_inbred(
  crosses,
  marker.mat,
  marker.effects,
  weights = NULL,
  nthreads = 4L
)
```

Arguments

crosses	A matrix or data.frame specifying the parents for each proposed cross: • two-way crosses: $n_crosses \times 2$ • four-way crosses: $n_crosses \times 4$ Entries may be either integer indices referring to rows of <code>marker.mat</code> (1-based), or character identifiers matching <code>rownames(marker.mat)</code>.
marker.mat	Numeric marker matrix with genotypes in rows and markers in columns. The coding must be consistent with the marker effects in <code>effects</code> (e.g., dosage coding).
marker.effects	Numeric matrix of marker effects with markers in rows and traits in columns. Must have <code>nrow(effects) == ncol(marker.mat)</code> .
weights	Optional numeric vector of trait weights of length <code>ncol(effects)</code> . If provided, an index value <code>IDX</code> is computed for each cross as a weighted sum of predicted EGEBVs across traits.
nthreads	Integer (default 4). Number of threads used by the C++ backend.

Details

This function supports two-way crosses (2 parents) and four-way crosses (4 parents).

Value

If `weights` is `NULL`, returns a data.frame with columns:

- parent identifiers (`parent1`, `parent2`; optionally `parent3`, `parent4`)
- predicted cross means per trait: `EGEBV1`, `EGEBV2`, ...

If `weights` is provided, returns a list with:

- `cross.df`: the data.frame described above
- `index.df`: a data.frame with parent identifiers and a single column `IDX`

calc_midparent_outcross

Predicted outcross means from specified two-way crosses (EGEBV and ETGV)

Description

Compute predicted cross means for two-way (bi-parental) crosses using marker effects. For each cross and each trait, the function returns:

- GEBV: the mid-parent mean of parental additive genomic values.
- TGV: the predicted mean total genotypic value of the F1, combining additive and (optional) dominance contributions (Werner et al).

If weights are provided, two index values are also returned: a weighted index based on the EGEBV trait means and a weighted index based on the ETGV trait means.

Usage

```
calc_midparent_outcross(
  crosses,
  marker.mat,
  marker.effects.A,
  marker.effects.D = NULL,
  weights = NULL,
  nthreads = 4L
)
```

Arguments

crosses	A matrix or data.frame (n_crosses x 2) specifying the parents for each proposed cross. Entries may be either integer indices referring to rows of marker.mat, or character identifiers matching rownames(marker.mat).
marker.mat	Numeric marker dosage matrix with genotypes in rows and markers in columns. The coding must be consistent with the marker effects provided in effects.A and effects.D.
marker.effects.A	Numeric matrix of additive marker effects (markers in rows, traits in columns). Must satisfy nrow(marker.effects.A) == ncol(marker.mat).
marker.effects.D	Optional numeric matrix of dominance marker effects (markers in rows, traits in columns). Must have the same dimensions as marker.effects.A. If NULL, dominance is ignored (treated as zero).
weights	Optional numeric vector of trait weights of length ncol(marker.effects.A). If provided, index values GEBV.IDX and TGV.IDX are computed per cross as weighted sums across traits.
nthreads	Integer (default 4). Number of threads used by the C++ backend.

Value

If `weights` is `NULL`, a `data.frame` with columns:

- `parent1`, `parent2`
- predicted trait means: `EGEBV1..EGEBVt`, `ETGV1..ETGVt`

If `weights` is provided, a list with:

- `cross.df`: the `data.frame` described above
- `index.df`: a `data.frame` with `parent1`, `parent2`, `GEBV.IDX`, `TGV.IDX`

`calc_optimal_haploid_value`

Optimal Haploid Value (OHV) for proposed crosses

Description

Computes the Optimal Haploid Value (OHV; Daetwyler et al.) for each proposed cross using block-wise local genomic estimated breeding values (GEBVs).

Usage

```
calc_optimal_haploid_value(
  crosses,
  marker.mat,
  marker.effects,
  weights = NULL,
  haplotype.blocks = NULL,
  nthreads = 4L
)
```

Arguments

<code>crosses</code>	A matrix or <code>data.frame</code> (<code>n_crosses</code> x 2 for two way crosses or <code>n_crosses</code> x 4 for four way crosses) specifying the parents for each proposed cross. Entries may be either: • integer indices referring to rows of <code>marker.mat</code>, or • character identifiers matching <code>rownames(marker.mat)</code>.
<code>marker.mat</code>	Numeric genotype or marker matrix with individuals in rows and markers in columns.
<code>marker.effects</code>	Numeric matrix of marker effects with <code>nrow(effects) == ncol(marker.mat)</code> with traits in columns.
<code>weights</code>	Optional numeric vector of length <code>ncol(effects)</code> . If supplied, a weighted index is computed as a linear combination of the trait-specific OHVs.
<code>haplotype.blocks</code>	Optional <code>data.frame</code> defining haplotype blocks with columns: • <code>block</code>: block identifier (integer, character, or factor). Markers sharing the same value belong to the same haplotype block. • <code>site</code>: marker identifier within <code>marker.mat</code>, given either as an integer column index (<code>1..ncol(marker.mat)</code>) or as a marker name matching <code>names(marker.mat)</code>. Each marker may appear in at most one block.
<code>nthreads</code>	Integer ≥ 1 . Number of threads used for parallel computation.

Details

For each haplotype block, a local GEBV is calculated for every parent as the sum of marker genotypes within the block weighted by their corresponding marker effects. The OHV of a cross is then obtained by summing, across blocks, the maximum of the two parents' local block GEBVs.

If `haplotype.blocks` is not supplied, each marker column of `marker.mat` is treated as an independent block.

Value

If `weights` is `NULL`, a `data.frame` containing the cross definition columns followed by `OHV.1`, `OHV.2`, ... for each trait.

If `weights` is supplied, a list with elements:

- `cross.df`: `data.frame` of crosses and trait-specific OHVs
- `index.df`: `data.frame` of crosses and the weighted OHV index

calc_spv_inbred	<i>Segregation variance for inbred lines from specified crosses</i>
-----------------	---

Description

Calculate expected genomic estimated breeding values (EGEBV), segregation variance, and superior progeny value (SPV) for doubled haploid or recombinant inbred lines #*t* derived after *t* rounds of random mating. works for two way, three way and four way crosses

Usage

```
calc_spv_inbred(
  crosses,
  genetic.map,
  marker.mat,
  marker.effects,
  t,
  intensity = NULL,
  type = "DH",
  weights = NULL,
  covariance = FALSE,
  method = 1,
  nthreads = 4L
)
```

Arguments

<code>crosses</code>	A matrix or <code>data.frame</code> (<code>n_crosses</code> x 2 for two way crosses or <code>n_crosses</code> x 4 for four way crosses) specifying the parents for each proposed cross. Entries may be either: • integer indices referring to rows of <code>marker.mat</code>, or • character identifiers matching <code>rownames(marker.mat)</code>.
<code>genetic.map</code>	A <code>data.frame</code> with columns:

	• site: integer marker index (1..ncol(marker.mat)) • chr: chromosome identifier (numeric) • pos: position on the chromosome in morgan All markers in marker.mat must appear in genetic.map\$site.
marker.mat	Numeric marker matrix (markers in columns, genotypes in rows) with entries in c(0, 2) corresponding to c("AA", "BB").
marker.effects	Numeric matrix of marker effects (markers in rows, traits in columns). Must have nrow(effects) == ncol(marker.mat).
t	Integer. Number of random-mating generations before DH or RIL creation.
intensity	Double. Standardized selection differential, defaults to 1.
type	intended offspring typew can be "RIL" for recombinant inbred lines or "DH" for double haploid lines.
weights	Numeric vector of length ncol(marker.effects) with fixed trait weights. If supplied the function calculates index values for each cross.
covariance	Logical. If TRUE, also compute the segregation covariance matrix between all supplied traits.
method	Character. Which method to use; one of c(1, 2) for Lehermeier or Osthusen-rich.
nthreads	Integer (default 4). Number of OpenMP threads (if enabled at compile time).

Value

If covariance = TRUE, a list with element cross.df (a data.frame) whose columns are named EGEBV1, var1, SPV1, EGEBV2, ... and a list with element covariances. If covariance = TRUE and calculate.index = TRUE, additionally an element index.df (a data.frame) with IDX, VAR.IDX, SPV.IDX. If covariance = FALSE, a data.frame with the same columns EGEBV1, var1, SPV1, EGEBV2,

calc_spv_outcross	<i>Additive and dominance segregation variance of families from specified crosses (Wolfe)</i>
-------------------	---

Description

Calculate expected genomic estimated breeding value (EGBV), expected total genetic value (ETGV), additive and dominance segregation variance, superior progeny value with additive variance (SPV), and superior progeny value including dominance variance (TSPV) for F1 families.

Usage

```
calc_spv_outcross(
  crosses,
  genetic.map,
  hap.mat1,
  hap.mat2,
  marker.effects.A,
  marker.effects.D,
  intensity = NULL,
```

```

weights = NULL,
covariance = FALSE,
nthreads = 4L
)

```

Arguments

<code>crosses</code>	A data.frame or matrix (n_crosses x 2) of parental indices (row indices into hap.mat1/hap.mat2) specifying the crosses.
<code>genetic.map</code>	A data.frame with columns: • <code>site</code>: integer marker index (1..ncol(hap.mat1)) • <code>chr</code>: chromosome identifier (numeric or factor) • <code>pos</code>: position on the chromosome in morgan All markers in hap.mat1/hap.mat2 must appear in genetic.map\$site.
<code>hap.mat1</code>	Numeric haplotype matrix (individuals x markers) for the first haplotype, entries in c(0, 1) for c("A", "B").
<code>hap.mat2</code>	Numeric haplotype matrix (individuals x markers) for the second haplotype, entries in c(0, 1) for c("A", "B").
<code>marker.effects.A</code>	Numeric matrix of additive marker effects (markers in rows, traits in columns). Must have nrow(marker.effects.A) == ncol(hap.mat1).
<code>marker.effects.D</code>	Numeric matrix of dominance marker effects (markers in rows, traits in columns). Must have nrow(marker.effects.D) == ncol(hap.mat1) and ncol(marker.effects.D) == ncol(marker.effects.A).
<code>intensity</code>	Double. Standardized selection differential (used for SPV), defaults to 1.
<code>weights</code>	Numeric vector of length ncol(marker.effects.A) with fixed trait weights. If supplied the function calculates index values for each cross.
<code>covariance</code>	Logical. If TRUE, also compute the segregation covariance matrix between all supplied traits.
<code>nthreads</code>	Integer (default 4). Number of OpenMP threads (if enabled at compile time).

Value

If `covariance = TRUE`, a list with element `cross.df` (a data.frame) whose columns are named GEBV1, TGV1, var.A1, SPV1, var.D1, TSPV1, EGEBV2, ... and covariance matrices per cross. If `weights` are supplied, additionally an element `index.df` (a data.frame) with `IDX.A`, `var.IDX.A`, `SPV.IDX`, `IDX.AD`, `var.IDX.AD`, `SPV.IDX.AD`. If no `weights` are supplied, a data.frame with the same columns.

make_cross_plan

Creation of a plan of all potential crosses

Description

Generates a crossing plan containing all possible pairwise crosses among a set of candidate genotypes. The function supports both unsexed (symmetric) crossing designs and sexed designs with distinct male and female candidate sets.

Usage

```
make_cross_plan(
  parents = NULL,
  self = FALSE,
  male.parents = NULL,
  female.parents = NULL
)
```

Arguments

parents	A vector of genotype identifiers defining the candidate parents for an unsexed crossing design. Identifiers may be integers or character strings (for example, row names of a genotype matrix). Must contain at least two unique entries.
self	Logical. If TRUE and parents is supplied, self-crosses (parent $\times$ itself) are included in addition to all pairwise crosses. Ignored when male.parents and female.parents are supplied.
male.parents	A vector of genotype identifiers defining male parents in a sexed crossing design. Identifiers may be integers or character strings. Ignored if parents is supplied.
female.parents	A vector of genotype identifiers defining female parents in a sexed crossing design. Identifiers may be integers or character strings. Ignored if parents is supplied.

Details

If parents is supplied, all unique unordered pairs of parents are returned. Optionally, self-crosses can be included by setting self = TRUE.

Alternatively, if male.parents and female.parents are supplied, all possible male–female combinations are returned. In this case, the self argument is ignored.

Value

A two-column data.frame defining the crossing plan.

- If parents is supplied, the columns are named parent1 and parent2 and contain all unique pairwise combinations (and optionally self-crosses).
- If male.parents and female.parents are supplied, the columns are named male and female and contain all male–female combinations.

The output preserves the type of the supplied identifiers (integer or character), with factors automatically coerced to character.

make_index

Calculation of a selection index

Description

Calculates a multi-trait selection index from genomic or phenotypic BLUPs. The function supports two alternative index formulations:

Usage

```
make_index(
  genotype.effects,
  weights = NULL,
  var.mat = NULL,
  desired.gains = NULL,
  nthreads = 4L
)
```

Arguments

- | | |
|------------------|---|
| genotype.effects | A numeric matrix ($n_{genotype} \times n_{trait}$) of multi-trait BLUPs. |
| weights | Numeric vector of length n_{trait} specifying economic weights for each trait. Used only for the Smith–Hazel index. |
| var.mat | Optional variance or covariance matrix used for the desired gains index. If provided, must be one of: <ul style="list-style-type: none"> • A numeric matrix of dimension $(n_{genotype} \times n_{trait}) \times (n_{genotype} \times n_{trait})$ containing the posterior variance $\text{var}(\tilde{g})$ of the stacked BLUPs. In this case, the <i>marginal</i> posterior trait covariance (obtained by averaging per-genotype trait blocks; Werner et al.) is used for index calculation. • A numeric matrix of dimension $n_{trait} \times n_{trait}$ giving the trait covariance matrix directly. If var.mat is NULL, the trait covariance matrix is estimated empirically as <code>cov(effects)</code> . |
| desired.gains | Numeric vector of length n_{trait} specifying the desired gains for each trait. Used only for the desired gains index. |
| nthreads | Integer. Number of OpenMP threads to use. |

Details

- **Desired gains index** , based on user-specified desired trait gains and a trait (co)variance matrix.
- **Smith–Hazel index**, based on user-specified economic weights.

Exactly one of gains or weights must be provided.

Value

A list with components:

- `index`: A data.frame containing the selection index values for each genotype.
- `weights`: The trait weights used to construct the index.

optimize_cross_plan	<i>Optimal cross selection via a genetic algorithm (with optional fixed crosses)</i>
---------------------	--

Description

Selects a mating plan consisting of `ncrosses` parent pairs by balancing a cross criterion (`criterion`) against a genomic similarity (or inbreeding proxy) taken from `G.mat`. The optimization can be run in one of two modes:

`method = "angle"` Returns a single mating plan that targets a user-specified trade-off between cross criterion and similarity, expressed as `target.angle` (0–90 degrees). Smaller angles prioritize cross criterion, larger angles prioritize lower similarity (more diversity).

`method = "pareto"` Returns an approximation of the Pareto frontier: a set of nondominated mating plans where no plan has both (i) lower cross criterion and (ii) higher similarity than another. This mode is more computationally demanding but provides multiple optimal trade-offs.

Usage

```
optimize_cross_plan(
  candidate.crosses,
  fixed.crosses = NULL,
  ncrosses,
  target.angle = 0,
  criterion,
  criterion.fixed = NULL,
  G.mat,
  method = "pareto",
  return.params = FALSE,
  plot = TRUE,
  params = list(),
  nthreads = 4L
)
```

Arguments

<code>candidate.crosses</code>	Integer or character matrix/data.frame with two columns ($K \times 2$). Candidate (variable) crosses. Must have exactly two columns; self-crosses are not allowed.
<code>fixed.crosses</code>	Integer or character matrix/data.frame with two columns ($F \times 2$) or NULL. Crosses that are always included in the final plan. Can have 0 rows. Must use the same indexing/identifiers as <code>candidate.crosses</code> . Self-crosses are not allowed.
<code>ncrosses</code>	Integer. Total number of crosses in the final plan (fixed + variable). Must be $\geq \text{nrow}(\text{fixed.crosses})$.
<code>target.angle</code>	Numeric. Target trade-off angle in degrees (0–90) used only when <code>method = "angle"</code> . Use 0 to prioritize cross criterion only and 90 to prioritize minimizing similarity (maximizing diversity).
<code>criterion</code>	Numeric vector of length <code>nrow(candidate.crosses)</code> . cross criterion for each candidate cross.

<code>criterion.fixed</code>	Numeric vector of length <code>nrow(fixed.crosses)</code> (or <code>NULL</code>). cross criterion for each fixed cross. If <code>fixed.crosses</code> has 0 rows, this can be length 0.
<code>G.mat</code>	Numeric square matrix ($n \times n$). Genomic relationship matrix used to compute similarity/inbreeding of parental contributions. If <code>candidate.crosses</code> or <code>fixed.crosses</code> are character, <code>rownames(G.mat)</code> must be present.
<code>method</code>	Character string, either "angle" or "pareto".
<code>return.params</code>	Logical. If TRUE, return additional diagnostics (see below).
<code>plot</code>	Logical. If TRUE and <code>method = "pareto"</code> , plot the estimated Pareto frontier. Ignored when <code>method = "angle"</code> .
<code>params</code>	List of GA parameters: <code>propability.mutate</code> Numeric in $[0, 1]$. Mutation probability per offspring. <code>n.mutate</code> Integer ≥ 0 . Number of positions to mutate when mutation occurs. <code>n.select</code> Integer ≥ 2 . Number of parents selected each generation. <code>n.pop</code> Integer ≥ 2 . Population size. <code>max.generation</code> Integer ≥ 1 . Maximum number of generations. <code>max.iteration</code> Integer ≥ 1 . Stop after this many generations without improvement. <code>angle.penalty</code> Numeric ≥ 0 . Penalty strength for deviating from <code>target.angle</code> (used only when <code>method = "angle"</code>).
<code>nthreads</code>	Integer ≥ 1 . Number of threads to use.

Details

If `fixed.crosses` is supplied, these crosses are always included in every returned plan. The GA then selects the remaining `ncrosses - nrow(fixed.crosses)` crosses from `crosses`.

Both `crosses` and `fixed.crosses` can be provided either as 1-based integer indices (referring to rows/columns of `G.mat`) or as character identifiers matching `rownames(G.mat)`.

Value

If `return.params = FALSE`:

- A data.frame with columns `parent1`, `parent2` describing the selected mating plan.

If `return.params = TRUE` and `method = "angle"`:

- `crossPlan`: selected plan (data.frame with `parent1`, `parent2`)
- `uMax`, `simMax`: cross criterion optimum and its similarity
- `uBest`, `simBest`: achieved solution
- `angleBest`, `lenBest`: objective-space angle/length diagnostics

If `return.params = TRUE` and `method = "pareto"`:

- `pareto.plans`: list of mating plans (each a data.frame with `parent1`, `parent2`)
- `pareto.frontier`: data.frame with columns `pareto.id`, `u`, `sim`

plot_cross_plan	<i>Plot cross plan ridge distributions per trait</i>
-----------------	--

Description

Creates ridgeline density plots of simulated cross outcomes, separated by trait. For each cross and trait, values are sampled from a Normal distribution whose mean is taken from the corresponding predicted value column and whose variance is taken from the available variance components.

Usage

```
plot_cross_plan(cross.plan, cross.df, traits = NULL, nsamples = 1000L)
```

Arguments

cross.plan	data.frame with 2 columns (2-way crosses) or 4 columns (4-way crosses), defining the parents in each cross. Rows are used to label and order crosses in the plot.
cross.df	data.frame (or object coercible to a data.frame) as created from the <code>get_variance</code> or <code>get_optimal_haploid_value</code> function containing per-cross trait columns such as GEBV#, TGV#, SPV#, TSPV#, OHV#, and variance components var#, var.A#, var.D#. If cross.df contains parent columns (e.g., parent1..parent4 or male, female), they are used to align rows to crosses.
traits	integer vector of trait indices to plot (e.g., <code>c(1,3)</code>), or NULL to plot all traits found in cross.df.
nsamples	integer. Number of Monte Carlo samples per cross and trait.

Details

Variance / mean handling (per trait):

- If var.A# is available, an *additive* ridge is drawn using mean = GEBV# and sd = sqrt(var.A#).
- If var.A# and var.D# and TGV# are available, an additional *additive+dominance* ridge is drawn using.
- If only var# is available (and no var.A#), a single ridge is drawn using mean = GEBV# and sd = sqrt(var#).

Point overlays are added when the corresponding columns are present:

- GEBV# (always used when available) and SPV# are shown as points and share the additive colour family.
- TGV# and TSPV# are shown as points and share the additive+dominance colour family.
- OHV# is shown as a black cross (shape = 4).

A dotted vertical line indicates the per-trait average gain (mean of GEBV# across crosses). The plot is faceted by trait (up to 3 columns).

Value

A ggplot object.

predict_response	<i>Summarize a multi-trait selection index</i>
------------------	--

Description

Computes summary statistics for a linear selection index given a trait covariance matrix and either (i) desired gains or (ii) fixed weights (Smith-Hazel).

Usage

```
predict_response(
  var.mat = NULL,
  desired.gains = NULL,
  weights = NULL,
  intensity = 1,
  plot = TRUE
)
```

Arguments

var.mat	Numeric trait covariance matrix ($n_{\text{trait}} \times n_{\text{trait}}$).
desired.gains	Optional numeric vector of desired gains (length n_{trait}). If supplied, index weights are computed as <code>solve(var.mat) %*% gains</code> .
weights	Optional numeric vector of index weights (length n_{trait}). Supply this to summarize an index with fixed weights. Using <code>cov(EBVs)</code> (i.e. <code>cov(BLUPs)</code>) as <code>var.mat</code> provides an empirical implementation of the Henderson form of the Smith-Hazel index.
intensity	Numeric scalar. Standardized selection differential i used to scale expected responses (default 1).
plot	Logical. If TRUE, produce a bar plot of expected trait gains (default TRUE).

Details

For a trait covariance matrix G and index weights b , the index is $I = b^\top T$. The function returns the index variance $\sigma_I^2 = b^\top G b$, the expected standardized response in the index $i\sigma_I$ (where i is the standardized selection differential), the expected response in each trait iGb/σ_I , and the correlation between each trait and the index.

If gains are provided, weights are computed as $b = G^{-1}d$, where d is the desired gains vector.

Value

A list with:

- `overall.df`: data.frame with `index.var` (σ_I^2) and `gain.index` ($i\sigma_I$)
- `traits.df`: data.frame with per-trait variance, correlation with the index, expected gain, and weight

summarize_cross_plan *Summarize the expected performance of a cross plan*

Description

Computes the average predicted performance of all crosses in a cross plan for each available trait and value metric (e.g. GEBV, TGV, SPV, TSPV, OHV). Crosses in `cross.plan` are matched to `cross.df` using their parental identities.

Usage

```
summarize_cross_plan(
  cross.plan,
  cross.df,
  plot = T,
  reference.pop = NA,
  weights = NA
)
```

Arguments

<code>cross.plan</code>	A data.frame with 2 columns (2-way crosses) or 4 columns (4-way crosses).
<code>cross.df</code>	A data.frame (or coercible to one) containing per-cross predictions. Trait columns should be named as <code><metric>.<trait></code> (e.g. <code>GEBV.Yield</code> , <code>SPV.Height</code>). If parent identifier columns are present (<code>parent1</code> – <code>parent4</code> or <code>male/female</code>), they are used to align crosses.
<code>plot</code>	Logical; if TRUE, plots the predicted mean GEBV response.
<code>reference.pop</code>	Optional reference population with individuals in rows and traits in columns. Used to express responses relative to the reference population mean.
<code>weights</code>	Optional vector of trait weights used to calculate and display the desired response.

Details

Optionally, the predicted population response (mean GEBV) can be plotted. If a reference population is supplied, responses are shown relative to the reference mean. If trait weights are also provided, the plot additionally displays the response expected under the desired response.

Value

A data.frame with one row per trait containing the average value of each available metric (e.g. `mean.GEBV`, `mean.TGV`, `mean.SPV`, `mean.TSPV`, `mean.OHV`) and the number of crosses (`ncrosses`).

Index

`backsolve_marker_effects`, [2](#)

`calc_midparent_inbred`, [3](#)
`calc_midparent_outcross`, [4](#)
`calc_optimal_haploid_value`, [5](#)
`calc_spv_inbred`, [6](#)
`calc_spv_outcross`, [7](#)

`make_cross_plan`, [8](#)
`make_index`, [9](#)

`optimize_cross_plan`, [11](#)

`plot_cross_plan`, [13](#)
`predict_response`, [14](#)

`summarize_cross_plan`, [15](#)